Trabajo Primer Corte

Integrantes:
Luis Carlos Bucheli Prado
Leidy Vanessa Pérez Zúñiga

Laboratorio de Programación Orientada a Objetos

Docente:
Rubén Darío Vargas Yandy

Universidad del Cauca
Facultad de ingeniería Electrónica y Telecomunicaciones- FIET
2026
Popayán Cauca

1. **Modelo de datos y estrategia de consistencia**

**1.1 Clases**

| Clase | Responsabilidad | Atributos Principales | Relaciones |
|---|---|---|---|
| Autor | Datos del autor | nombre,nacionalidad, fechaNacimiento, fechaFallecimiento, librosPublicados | Referenciado por libro |
| Libro | Información catalográfica | Titulo, year, isbn | Tiene un Autor* |
| Ejemplar | Existencia física de un libro en una estantería | cantidadDisponible, prestamos | Tiene un libro*, un Estante* y varios Prestamo* |
| Estante | Agrupa ejemplares | nombre | Tiene varios Ejemplar* y un Biblioteca* |
| Biblioteca | Agrupa estantes | nombre | Tiene varios Estante* |
| Estudiantes | Persona que recibe prestamos | Codigo,nombre,carrera | Referenciado por Prestamos |
| Préstamo | Registro de prestamos | activo | Tiene un Ejemplar* y un Estudiante* |

**1.2 Relaciones**

```
Biblioteca 1 ---- * Estante 1 ---- * Ejemplar * ---- 1 Libro * ---- 1
Autor
                                        |
                                        * Prestamo * ---- 1 Estudiante
```

**1. 3 Separación entre catalogo y existencia**

Libro guarda solo título, autor, año e ISBN. Ejemplar guarda la cantidad disponible y su ubicación (estante).Así , un mismo Libro puede tener varios Ejemplar (uno por estante o biblioteca) con cantidades independientes, sin duplicar el catalogo.

**1.4 Estrategia de consistencia**

**Decisión:** asociación por punteros y un único objeto Ejemplar por cada par(libro,estante).

- Cada Ejemplar se crea una sola vez con new y su dirección se guarda en el vector<Ejemplar*> del Estante.
- Todas las operaciones (consultar, listar, contar, prestar, actualizar) llegan al mismo objeto por ese puntero, sin copias.
- Si se cambia cantidadDisponible(por actualización o por préstamo), cualquier consulta posterior desde otra parte del sistema ve el valor nuevo.
- Libro y Autor se comparten igual: los ejemplares apuntan al mismo Libro, y este al mismo Autor.

**Por qué no por valor:** si Estante guardara vector<Ejemplar>, o se modificara una copia, el cambio no llegaría al original y habría inconsistencia.

*los getters(getEjemplares, getEstantes, getPrestamos) devuelven una copia del vector de punteros, no de los objetos. Por eso los punteros siguen apuntando a los mismos ejemplares .*

### 1.5 Propiedad de la memoria

| Quien | Es dueño de | Quien lo libera |
|---|---|---|
| main(listas globales) | Autor, Libro,Estudiante, Biblioteca | LiberarMemoria() |
| Biblioteca | Sus Estante | ~Biblioteca () |
| Estante | Sus Ejemplar | ~Estante() |
| Ejemplar | Sus Prestamo | ~Ejemplar() |

Al borrar una biblioteca se libera en cascada todo lo que contiene. Libro, Autor y Estudiante no se borran en cascada porque son compartidos por punteros y tienen su propio dueño en las listas globales.

## 2. Interfaz de operación

El main () solo muestra el menú, lee la opción y delega en una función por proceso.

| Opción | Proceso | Función | Que hace |
|---|---|---|---|
| 1 | Registro de autor | registrarAutor() | Pide datos y guarda el Autor en listaAutores |
| 2 | Registro de libro | registrarLibro() | Pide datos y asocia un autor ya registrado |
| 3 | Registro de biblioteca | registrarBiblioteca() | Crea la Biblioteca |
| 4 | Registro de estantería y asignación a biblioteca | registrarEstante() | Crea el Estante y lo agrega a la biblioteca elegida |
| 5 | Registro de ejemplar | registrarEjemplar() | Elige libro, biblioteca y estante; crea el Ejemplar con su cantidad |
| 6 | Consulta de disponibilidad | consultarDisponibilidadLibro() → Biblioteca::consultarDisponibilidadLibro() | Recorre todas las bibliotecas y muestra biblioteca, estante y cantidad |

| | | | |
|---|---|---|---|
| 7 | Inventario y conteo | listarInventarioBiblioteca() → listarInventario() y contarTotalEjemplares() | Lista libro, estante y cantidad, y el total |
| 8 | Actualización de inventario | actualizarCantidadEjemplar() | Busca ejemplares por título; si hay varios, se elige cuál; cambia su cantidad |
| 9 | Registrar estudiante | registrarEstudiante() | Crea el Estudiante |
| 10 | Registrar préstamo | registrarPrestamo() | Busca un ejemplar con unidades y lo presta |
| 11 | Préstamos activos | listarPrestamosActivos() → Biblioteca::listarPrestamosActivos() | Lista libro y estudiante de una biblioteca |
| 12 | Libros de un estudiante | verLibrosPrestadosEstudiante() | Lista los préstamos activos del estudiante |
| 0 | Salir | main() + liberarMemoria() | Termina y libera memoria |

**Lectura de datos:** leerLinea() (usa getline, permite espacios y campos vacíos) y leerEntero() (valida y vuelve a pedir si no es número).

**Validaciones:** listas vacías, opciones fuera de rango, cantidad negativa y entrada no numérica.

**3. Organización del código**

- **Una clase por concepto** del dominio, con atributos privados, constructores, getters y setters.
- **La lógica del dominio vive en las clases:** Biblioteca cuenta y lista, Ejemplar decide si hay disponibilidad y presta.
- **Funciones de interfaz** (registrarX, consultarX, listarX): piden datos, validan y delegan.
- **Funciones de apoyo:** leerLinea(), leerEntero(), buscarEjemplarParaPrestamo().
- **Función principal** main(): solo menú y switch.
- **Declaraciones anticipadas** para las dependencias circulares entre Ejemplar, Estante y Biblioteca.

**4. Gestión de préstamos**

Prestamo une un Ejemplar* con un Estudiante* y tiene un indicador activo. Cada Ejemplar guarda su vector<Prestamo*>.

| **Requisito** | **Implementación** |
|---|---|
| Registrar estudiante | registrarEstudiante() |
| Prestar un ejemplar disponible | registrarPrestamo() busca con buscarEjemplarParaPrestamo() el primer ejemplar del título con unidades y llama a Ejemplar::prestarAEstudiante(), que crea el Prestamo y descuenta 1 |
| Préstamos activos de una biblioteca | Biblioteca::listarPrestamosActivos() recorre estantes → ejemplares → préstamos activos |
| Libros prestados a un estudiante | verLibrosPrestadosEstudiante() filtra por estudiante y estaActivo() |

El descuento afecta al mismo Ejemplar, así que mantiene la consistencia del punto 1.

**Alcance:** no se implementó la devolución. finalizarPrestamo() existe en Prestamo pero no está en el menú porque el enunciado no la pide.

**5. Verificación**

**Compilación:** g++ -std=c++11 -Wall -Wextra -o biblioteca main.cpp → sin errores ni advertencias.

**Memoria:** ejecución con AddressSanitizer (-fsanitize=address) sin errores ni fugas.

| # | Acción (opción) | Datos de entrada | Resultado esperado | Resultado |
|---|---|---|---|---|
| 1 | Registrar autor (1) | Gabriel Garcia Marquez, Colombiano, 1927, fallecimiento vacío, 30 | Registrado y el programa continúa | Correcto |
| 2 | Registrar libro (2) | Cien anios de soledad, 1967, ISBN, autor 1 | Libro registrado | Correcto |
| 3 | Registrar bibliotecas (3) | Biblioteca Central, Biblioteca Facultad | Dos bibliotecas | Correcto |
| 4 | Registrar estantes (4) | Estante A (Central), Estante B (Facultad) | Dos estantes | Correcto |
| 5 | Registrar ejemplares (5) | Libro 1 en Estante A (3) y en Estante B (2) | Dos ejemplares | Correcto |
| 6 | Consultar disponibilidad (6) | Cien anios de soledad | Central/A: 3 y Facultad/B: 2 | Correcto |
| 7 | Registrar estudiante (9) | U001, Vane Perez, Ingenieria de Sistemas | Estudiante registrado | Correcto |
| 8 | Registrar préstamo (10) | Cien anios de soledad, estudiante 1 | Préstamo creado; Central baja a 2 | Correcto |
| 9 | Préstamos activos (11) | Biblioteca Central | Libro y estudiante | Correcto |
| 10 | Libros del estudiante (12) | Estudiante 1 | Libro prestado | Correcto |
| 11 | Inventario (7) | Biblioteca Central | Estante A: 2 y total 2 | Correcto |
| 12 | Actualizar cantidad (8) | Elegir Facultad/Estante B, nueva cantidad 7 | Consulta posterior muestra Facultad 7 y Central 2 | Correcto |
| 13 | Casos límite | Opción 99, letra en el menú, cantidad -1 | Mensaje de error sin cerrar | [por probar] |